



An anomaly-based approach for cyber–physical threat detection using network and sensor data

Roberto Canonico^{ID}, Giovanni Esposito, Annalisa Navarro^{ID}, Simon Pietro Romano^{ID},
Giancarlo Sperli^{ID*}, Andrea Vignali^{ID}

Department of Electrical Engineering and Information Technology (DIETI), University of Naples Federico II, Via Claudio 21, Naples, Italy

ARTICLE INFO

Keywords:

Threat detection
Anomaly detection
Unsupervised learning
ICS
CPS

ABSTRACT

Integrating physical and cyber realms, Cyber–Physical Systems (CPSs) expand the potential attack surface for intruders. Given their deployment in critical infrastructures like Industrial Control Systems (ICSs), ensuring robust security is imperative. Current research has developed various Intrusion Detection techniques to identify and counter malicious activities. However, traditional methods often encounter challenges in detecting several attack types due to reliance on a single data source such as time series data from sensors and actuators. In this study, we meticulously design advanced Deep Learning (DL) anomaly-based techniques trained on either sensor/actuator data or network traffic statistics in an unsupervised setting. We evaluate these techniques on network and physical data collected concurrently from a real-world CPS. Through meticulous hyperparameter tuning, we identify the optimal parameters for each model and compare their efficiency and effectiveness in detecting different types of attacks. In addition to demonstrating superior performance compared to various baselines, we showcase the best model for each data source. Eventually, we show how utilizing diverse data sources can enhance cyber-threat detection, recognizing different kinds of attacks.

1. Introduction

In recent years, Cyber–Physical Systems (CPSs) have undergone significant transformations in the industrial landscape, tightly integrating communication, computational, and physical capabilities [1]. However, with the pervasive diffusion of advanced Information and Communication Technologies (ICT) into Industrial Control Systems (ICS) and the growing complexity of their inherited architectures, various security challenges have arisen [2–7]. These challenges have led to an increase in the attack surface that may be exploited by malicious entities both inside and outside organizations, as well as across system boundaries [8]. As a result, intruders may perform malicious activities by sabotaging physical and/or virtual ICS components [9–11], or establishing a foothold on the organization [12,13].

In support of this, it is estimated that the security market in ICS will experience substantial growth in the forthcoming years, reaching

a market value of approximately \$30 billion by 2032.¹ This growth is driven by the continuous demand for new methodologies to cope with the increasing costs of cyber-attacks, which are estimated to be around \$4.72 million per incident in the energy sector and \$2.07 million per data breach in the governmental public sector in 2022.²

The past few years have witnessed a rise in attacks on ICSs [14,15]. Notable instances include the advanced ransomware attack on the Colonial Pipeline,³ the Triton Malware,⁴ which targeted Safe Instrument Systems in the ICS, and Stuxnet,⁵ a sophisticated cyber weapon that targeted Iran's nuclear facilities and posed severe risks to human lives. These incidents demonstrate the alarming potential of cyber-attacks in causing physical damage to critical infrastructures.

Therefore, extensive research on cyber–physical security has been conducted to detect cyber–physical attacks and improve ICS resilience

* Corresponding author.

E-mail addresses: roberto.canonico@unina.it (R. Canonico), giovanni.esposito54@studenti.unina.it (G. Esposito), annalisa.navarro@unina.it (A. Navarro), spromano@unina.it (S.P. Romano), giancarlo.sperli@unina.it (G. Sperli), andrea.vignali@unina.it (A. Vignali).

¹ https://www.gminsights.com/industry-analysis/industrial-control-systems-ics-security-market?utm_source=globenewswire.com&utm_medium=referral&utm_campaign=Paid_globenewswire

² <https://parachute.cloud/cyber-attack-statistics-data-and-trends/>

³ <https://www.bloomberg.com/profile/company/3205Z>

⁴ <https://www.cyberark.com/resources/threat-research-blog/anatomy-of-the-triton-malware-attack>

⁵ <https://edition.cnn.com/2011/11/08/tech/iran-stuxnet/index.html>

Acronyms	
Symbol	Description
AE	AutoEncoder
AE_{LSTM}	AutoEncoder based on LSTM
CIP	Common Industrial Protocol
CPS	Cyber-Physical Systems
DAS	Data Acquisition System
DL	Deep Learning
DLAD	Deep Learning Anomaly Detection
DPI	Deep Packet Inspection
GAN	Generative Adversarial Network
HMI	Human Machine Interface
IoT	Internet of Things
ICS	Industrial Control System
ICT	Information and Communication Technologies
IQR	Interquartile Range
LSTM	Long Short-Term Memory
PA	Point Adjustments
PLC	Programmable Logic Controller
RTU	Remote Terminal Unit
SWaT	Secure Water Treatment
TPR	True Positive Rate
TNR	True Negative Rate
VAE	Variational Autoencoder based on LSTM

[16], also considering the diagnosis and prognosis of faults [17]. However, identifying anomalies is becoming more challenging due to deceptive and stealthy attack behavior. Several techniques have been proposed to enhance ICS resilience. For instance, recent studies [18] suggest replicating ICS components, such as sensors or controllers, to achieve redundancy and functional diversity. While this approach theoretically improves resilience, it also significantly increases the attack surface.

In response to these challenges, practitioners and researchers have directed their efforts toward designing Deep Learning (DL) models [19] for cyber-attack detection in ICSs. Anomaly-based detection methods have been particularly proposed by researchers to identify malicious operations and cyber-attacks in ICSs [20]. This is because ICSs often exhibit consistent patterns in their behaviors [21], and once an ICS is established, its topology and communication protocols tend to remain fixed for an extended period of time.

We present a DL-based framework designed to enhance the resilience of CPS through intrusion detection. The framework utilizes a joint analysis of network and physical data, enabling it to effectively handle various types of cyber-physical attacks. Additionally, we conduct a comparison of different DL models.

In Fig. 1 we show the Research Process Flowchart followed in this paper. We defined a research objective with the question “*Can the use of both physical and network data improve anomaly detection in CPSs?*”, supported by the following two Research Questions (RQs): (i) RQ1: “*Can an anomaly detection model jointly considering physical data and network data improve attack identification in a CPS?*”; (ii) RQ2: “*Which are the best models for detecting attacks based on both physical data and network data?*”. Following the steps depicted in the figure (and named after the main sections of the paper), we found convincing answers to the RQs and were able to achieve the research objective, as also highlighted in the “Results” section.

Below, we summarize the proposed contributions of this work.

1. We conduct a comprehensive evaluation of four unsupervised anomaly detection models: AutoEncoder (AE), AutoEncoder based on LSTM

(AE_{LSTM}), Variational Autoencoder based on LSTM (VAE), and Generative Adversarial Network (GAN). The evaluation is performed on either network or physical data. Additionally, we perform hyperparameter tuning and compare the results with two baselines, namely USAD and GDN.

2. We use a portion of a publicly available real-world ICS dataset, which includes data from both the physical environment (sensor and actuator values) and the network infrastructure (network traffic traces) collected in the same time period. This dataset contains data from both normal operation and cyber-attack conditions. To the best of our knowledge, this portion of the dataset has not been used in previous studies, and it uniquely allows us to conduct a comprehensive analysis by considering both data sources simultaneously.
3. In our experimental analysis, we demonstrate that each data source can identify a specific type of attack when paired with an appropriate model. Furthermore, we emphasize the importance of integrating multiple data sources for comprehensive threat detection, ensuring a more robust defense mechanism against a diverse range of potential threats.

The paper is organized as follows. In Section 2, we provide the background of security in CPSs and review related work on anomaly detection utilizing both physical and network data. Section 3 presents our proposed framework, starting with a high-level overview and then providing a detailed explanation of each component. The testbed and dataset characterization are described in Section 4, and Section 5 presents the experimental protocol, including a discussion of our original hyperparameter tuning method. Finally, in Section 7, we draw conclusions and discuss potential future research directions.

2. Literature review

2.1. Background and motivation

An ICS is composed of network-connected computers and physical processes, which consist of sensors, actuators, and a local computing element, such as a Programmable Logic Controller (PLC) or Remote Terminal Unit (RTU) [22]. These components are interconnected using various local networking protocols, such as CAN, Profibus, Modbus, or S7, while the central control unit is connected through protocols like TCP or wireless networks, facilitated by a Data Acquisition System (DAS). The central control node includes a master terminal unit responsible for applying control logic to the RTUs and offering management capabilities through a Human Machine Interface (HMI) [23]. In addition, ICS commonly includes a historian server to store data from the RTUs and an engineering workstation running SCADA software.

The architecture is summarized in Fig. 2. The SCADA workstation enables operators to monitor and control industrial processes by displaying sensor data, providing system alerts, and allowing commands to be sent to actuators. The HMI is the graphical interface that connects operators with the control system, facilitating interaction and control. The Historian Server stores data collected from field devices, maintaining historical records that can be used for process optimization. An RTU is a field device that interfaces with sensors and actuators, collecting data, processing it, and sending it to the control center while also executing control commands from the SCADA system. PLCs are located near the processes they control, executing pre-programmed logic based on sensor inputs to manage outputs like motors and valves. Sensors measure the physical parameters of the process and convert them into electrical signals for RTUs and PLCs. Actuators carry out physical actions based on control system commands. The term ‘Physical’ refers to the actual industrial operations, such as manufacturing, chemical processing, or power generation, that are being monitored and controlled by the ICS.

Recent advancements have introduced support for the Common Industrial Protocol (CIP) in SCADA and PLC components, enabling the

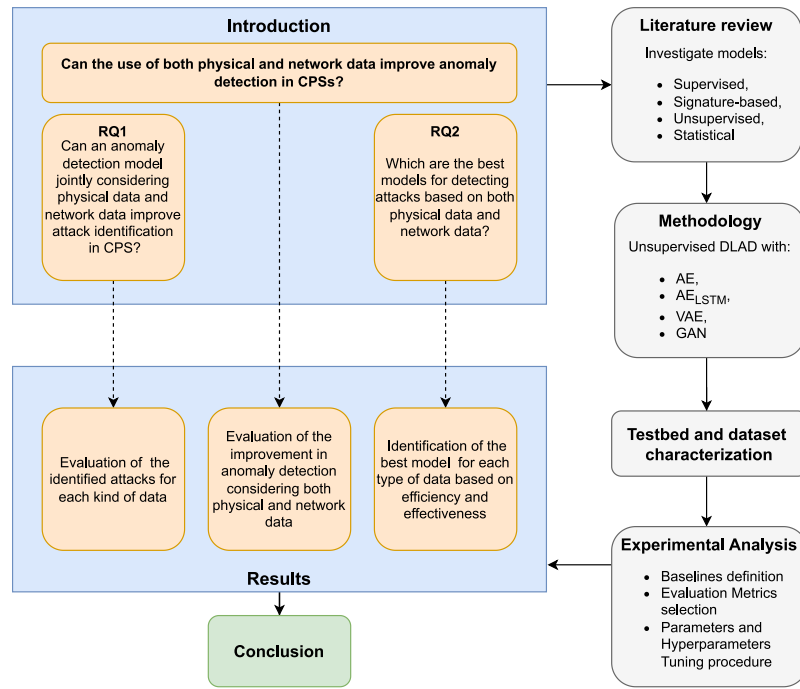


Fig. 1. Research Process Flowchart, in which we detail research questions, objectives, and the methodology we used to assess them.

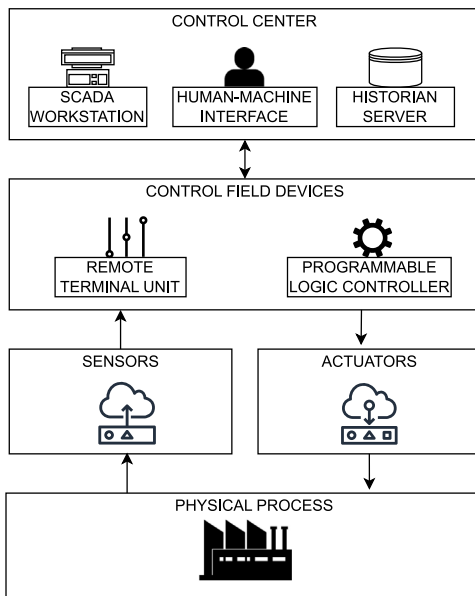


Fig. 2. ICSs expose a large attack surface. Attacks can be launched across all components and their interconnections (e.g., infecting the workstation, exfiltrating data from the historian server, compromising the PLC logic, tampering devices in the physical process, intercepting and modifying the signals from the sensors and actuators).

integration of industrial control applications with a standard network stack.

ICS is exposed to a significant attack surface, making it susceptible to attacks across all its components and interconnections [24]. An attacker could exploit vulnerabilities in the HMI or SCADA workstation, gain control of the control center, exfiltrate data, or launch DoS (Denial of Service) attacks on the historian server. The communication network is also at risk, with potential eavesdropping, replay, or false packet injection attacks [25]. PLCs are also potential targets, with attackers

exploiting software vulnerabilities or trust relationships between PLCs and SCADA systems to manipulate the PLC's logic. Additionally, sensors may be disrupted or replaced with malicious ones, and actuators' signals could be altered, causing significant disruptions in the physical processes [26]. Given the high criticality of ICSs and their extensive attack surface, it is crucial to prioritize the development of measures to enhance their security and protection.

2.2. Cyber-attack detection based on network data

Traditionally, a network anomaly detection framework consists of three main components [27]: traffic pre-processing, feature extraction, and anomaly detection. In the *traffic pre-processing phase*, data is collected from network middleboxes, and irrelevant packets (e.g., ICMP packets) that are not relevant for detection or may impact performance are filtered out. The *feature extraction phase* involves deriving feature vectors from the captured traffic data. Two types of features are commonly considered [28]: packet-level and flow-level features. Packet-level features, such as payload size, packet size, and payload ratio, are used to identify application anomalies. However, they may not always provide a distinct separation in values between malicious and legitimate traffic flows, affecting the accuracy of detection. On the other hand, flow-level features, such as mean packet length, focus on the characteristics of multiple packet sequences rather than the contents of individual packets. Finally, in the anomaly detection phase, algorithms are applied to determine whether the traffic exhibits suspicious behaviors.

There exist three primary approaches for detecting attacks using network traces [16,20]: machine learning-based approaches, anomaly detection, and signature-based detection.

Signature-based detection [29] relies on predefined attack signatures or patterns to identify known cyber-physical attacks. Network traffic is compared against a database of signatures, and if a match is found, an attack is detected. While this approach is effective against known attacks and benefits from an up-to-date signature database, it may struggle to detect new or zero-day attacks.

Machine learning techniques [30,31], including supervised learning and deep learning, has gained widespread adoption in cyber-physical

attack detection. These approaches involve training models with labeled network traffic data to identify patterns associated with attacks. Popular algorithms used in this context include Random Forest, Support Vector Machines (SVM), Naive Bayes, and Convolutional Neural Networks (CNN). The main challenges associated with supervised attack detection are the inability of the algorithm to detect zero-day attacks, which are previously unknown to the system, along with the lack of properly labeled attack datasets, making it difficult for the algorithm to distinguish between benign and malicious activities.

Anomaly detection methods [4] aim to detect deviations from normal network behavior, which may indicate the presence of an attack. Statistical techniques such as clustering, statistical modeling, and time series analysis are commonly employed to identify anomalies in network traffic. Additionally, techniques such as Principal Component Analysis (PCA) and Autoencoders are utilized to learn normal traffic patterns and identify deviations from them. This approach succeeds in detecting previously unknown attacks, including zero-day attacks, that cannot be detected by signature-based models [32].

2.3. Cyber-attack detection based on physical data

Cyber-attack detection based on anomaly detection using physical data from sensors and actuators is a critical area of research focused on protecting CPS and critical infrastructure [25]. The state-of-the-art approaches in this field revolve around leveraging the data collected from sensors and actuators to identify abnormal or anomalous behavior that may indicate the presence of cyber-attacks [4,33].

The detection of cyber-attacks based on anomaly detection using physical data involves the following key steps [4]: (i) *Data Collection*: Sensors and actuators embedded within the CPS continuously collect data on various physical parameters such as flow level and variation or the opening or closing of a valve. This data serves as the basis for cyber-attack detection by monitoring the normal operation of the system. (ii) *Capturing normal system behavior*: A baseline of normal system behavior is established using historical data or predefined models. This baseline represents the expected patterns and ranges of the physical data collected from the sensors and actuators during normal system operation. Statistical analysis and modeling techniques are commonly used to define the baseline. (iii) *Anomaly Detection*: Anomaly detection algorithms are applied to compare the real-time physical data collected from the sensors and actuators with the established baseline. Deviations or outliers that fall outside the normal range are flagged as potential anomalies, indicating the presence of a cyber-attack.

2.4. Comparison with the state of the art

As discussed in Section 2, there is growing importance in the design of robust security strategies to effectively address the devastating impact of cyber-physical attacks on interconnected systems. In light of this, recent literature has primarily focused on the implementation of Deep Learning Anomaly Detection (DLAD) models to detect such attacks within the cyber-physical context. In fact, modern attacks are becoming more sophisticated, targeting both sensor/actuator data and network traffic, with the aim to deceive monitoring platforms, posing novel and challenging issues, as discussed in [6]. Furthermore, CPS plants are often prone to different reconfigurations due to changes in operation sequences or adaptations to new manufacturing processes, as shown in [41]. Hence, new attacks or failures are introduced that require the use of DLAD techniques.

Although network anomaly detection approaches have been proposed, they often suffer from limitations such as being too specific, focusing on individual protocols, or making coarse estimations. For example, some literature examples include the use of periodicity detection in Ethernet for control automation technology (EtherCAT) networks [35], which may not address the immediacy of the attack,

and another study [37], which considers sessions longer than a certain threshold without fully considering the temporal complexities of cyber-physical attacks.

Furthermore, a significant challenge arises due to the diverse nature of attacks, which are becoming increasingly stealthy and can target multiple layers of a cyber-physical system's architecture, including the physical, control, and network layers [33,42]. The current state of the art is limited by its reliance on anomaly detection models that operate solely on a single data source [4].

Furthermore, some existing methods rely on Point Adjustments (PA) to detect anomalies within the dataset [34,36,43], but this approach can lead to delayed discovery of anomalies and potentially overestimate the algorithms' performance [44]. In real-world scenarios, it is essential to discard such an evaluation protocol and consider a more effective approach to evaluating the proposed methods since an anomaly remains as such even when considering a single sample. In Table 1 we analyze the pros and cons of recent cyber-attack detection methodologies ordered by year with the emphasis on the methodology and the data source. It is worth noticing that in Table 1 the listed approaches do not leverage both the network traffic source, represented in the form of biflow statistics, and the time series of the values collected from sensors/actuators to perform anomaly detection in CPS. While traditional approaches often rely on packet values extracted through techniques like Deep Packet Inspection (DPI) to identify SCADA commands, we utilize statistics from network traffic biflows as input for neural networks marking a departure from conventional CPS anomaly detection methodologies. Regarding the methodology, we refrain from employing techniques like PCA, which struggles to capture non-linear dependencies, and Periodicity detection, which may not promptly detect threats. Similarly, for network traffic analysis, we steer clear of signature-based methods that cannot identify new or unseen attacks, as well as heuristic clustering due to its lack of guarantees. Considering that ensemble methods encounter challenges such as hyperparameter tuning complexity, we turn to unsupervised deep learning-based anomaly detection methods to detect new or unseen threats, evaluating the effectiveness of *GAN*, *AE*, and their variants (such as *AE_{LSTM}* for capturing long temporal dependencies and *VAE* for statistical characteristics of the data distribution). While signature-based approaches and supervised learning approaches were also investigated, *GAN*, *AE*, *AE_{LSTM}* and *VAE* are chosen since they are all unsupervised methods with the ability to detect previously unseen attacks, such as zero-day attacks, by identifying deviations from normal behavior. We choose *AE*, *AE-LSTM*, *VAE*, and *GAN* because they are the most used models in the anomaly detection task under unsupervised settings, as shown in [43]. They also represent a diverse set of unsupervised anomaly detection techniques, each capable of capturing different aspects of the analyzed data, such as latent representations, temporal dependencies, and statistical characteristics.

Finally, we posit that an individual data source alone is insufficient for the comprehensive detection of various attacks. Instead, a more effective approach calls for the simultaneous analysis of network and sensor data sources. By combining these two types of data, a more comprehensive view can be obtained, enabling the identification of attacks that may otherwise go unnoticed when considering only a single data source. By leveraging both sources of information, security strategies can be designed to effectively detect and mitigate a wider range of attacks, thereby safeguarding the integrity and functionality of cyber-physical systems and their surrounding environments.

3. Methodology

The primary goal of anomaly detection is to identify anomalous sections in the data by leveraging the regular patterns present in the historical data. In our research, we used observations collected during normal working conditions to train the model. This allowed the model to learn and understand the system's working pattern by predicting

Table 1

Papers ordered by **Year**, focusing on anomaly detection in CPSs. The **Application** classifies the papers as *CPS* if they target CPS directly or use datasets from CPS; *Generic* otherwise. The **Dataset** indicates whether the data used is physical (*P*) or network data (*N*). (The full circle (●) denotes the use of network traffic statistics, while the half-empty circle (◐) denotes the utilization of SCADA-like commands). The **Methodology** describes the techniques used. The **Characteristics** outline the pros and cons of each methodology employed.

Ref.	Year	Application	Data		Methodology	Characteristics
			P	N		
[34]	2020	CPS	●		AE with adversarial training	+ Needs only unlabeled data for training - Noise in training data increases the number of FPs - Dependency on hyperparameters
[23]	2021	CPS	●		Principal Component Analysis (PCA)	+ Dimensionality Reduction - Sensitivity to outliers - Interpretability, Linearity Assumption
[35]	2021	CPS		●	Periodicity detection approach for CPS	+ Does not affect real-time communication and bandwidth + Generalization to similar vulnerabilities - May not address the immediacy of an attack
[5]	2021	Generic		◐	Ensemble of unsupervised machine learning	+ Needs only unlabeled data for training + Good generalization ability - Hyperparameter tuning complexity - Complexity in defining ensemble strategies
[36]	2021	CPS	●		Attention-based graph neural network approach	+ Ability to identify root causes + Good for recognizing complex relationships in data - High data dependency to structure the graph - Slow training times
[29]	2022	Generic		●	Signature-based	+ Ease of implementation - Not Recognizing New or Unseen Threats
[37]	2023	CPS		●	Heuristic Clustering	+ Simplicity, Explainability - Lack of Guarantees, Sensitivity to Parameters
[28]	2023	Generic		●	Machine Learning/Deep Learning (Unsupervised)	+ Detection of Unknown Threats - Data quality and quantity, Explainability
[38]	2023	CPS	●		GRU-Based Interpretable Multivariate Time Series Anomaly Detection	+ Detection of Unknown Threats + Fast detection, Interpretability - Capturing shorter term temporal dependencies
[39]	2023	Generic	●		Denoising AE	+ Robust against the heterogeneous environment of IoT - Does not capture Temporal Dependencies
[40]	2023	Generic		●	AE, Deep AE, AE ensemble	+ Fast Detection, Robust to poisonous attacks - Does not capture Temporal Dependencies
Our paper	2024	CPS	●	●	Unsupervised DLAD based on AE, AE_{LSTM} , AE_{VAE} , GAN	+ Detection of Unknown Threats + Fast detection, Fast training times + Captures Long Temporal and Statistical Dependencies

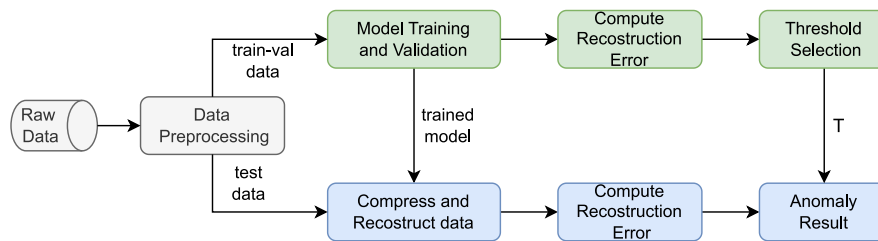


Fig. 3. Models train on normal data, learn working patterns, and use errors to set an anomaly threshold. In testing, the model predicts and reconstructs data, marking higher error observations above the threshold as anomalies.

future time series and reconstructing the original input. Once the model is trained, it is then used to detect anomalies in the test dataset that contains attack instances. Anomalous results are determined by comparing the error of the attack data against a predetermined threshold. Higher error values indicate a higher likelihood of anomalous behavior.

Our proposed approach consists of two distinct stages: the training phase and the testing phase. In the training phase, the model learns the working pattern of the ICS based solely on normal data. Once the model is sufficiently trained, the errors obtained during training with normal data are used to determine a threshold for identifying anomalies. In the testing phase, the performance of the trained model is evaluated

using data that contains both normal and attack instances. The model reconstructs and predicts the testing data that includes attacks and any observations with errors higher than the threshold are marked as anomalous (Fig. 3). The error is computed as the Mean Square Error (MSE) between the reconstructed outputs and the input.

3.1. Data pre-processing

In an ICS, data can be sampled from two distinct sources: sensors and actuators related to the physical infrastructure, and network packets associated with the network infrastructure. To effectively feed

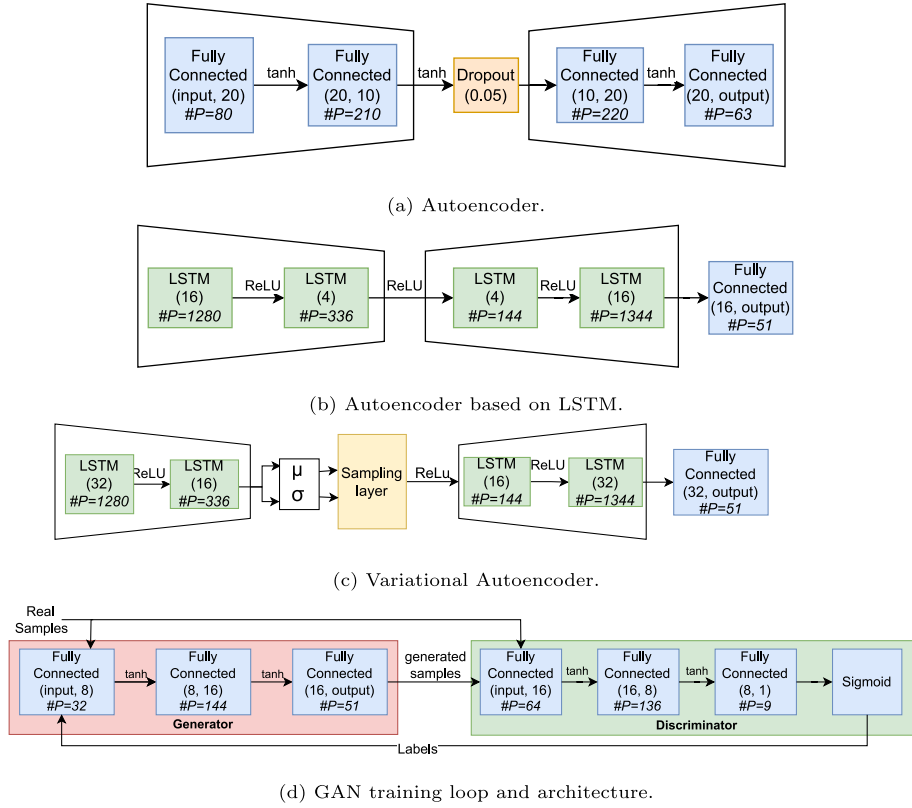


Fig. 4. Architecture of the different models. The Input and Output shapes ($n \times n$), the number of neurons (n) per layer, and the number of trainable parameters ($\#P$) per layer are reported.

the deep learning-based models for anomaly detection with proper information, these data sources must undergo different pre-processing steps.

Physical data. The data obtained from sensors and actuators require specific processing techniques, such as scaling, normalization, and filtering, to convert raw data into a usable format. On the other hand, network packets need various kinds of pre-processing operations, such as aggregation and conversion into relevant protocol data units. Recognizing the unique requirements of each type of data, the pre-processing stage can convert raw data into more structured and consistent information, facilitating the early detection of anomalous behavior.

At the same time, data in the form of multivariate time series are collected from the sensors and actuators of the ICS, constituting the physical features. The data are filtered by subsystem and then transformed using a *min-max* scaler.

Network data. Data is collected from the network traffic in the form of network *biflows*, which represent bi-directional network traffic flows between two communication endpoints. For each traffic biflow, the following features are extracted: duration, the total number of packets, the total length of packets, the number of bytes and packets per second, the packet length, and the interarrival time (mean, minimum, maximum, standard deviation, total) in both the forward and backward directions.

3.2. Anomaly detection models

The anomaly detection task has received extensive attention in the literature across various domains [19]. The proposed approach aims to enhance state-of-the-art solutions by capturing long temporal and statistical dependencies, as well as combining physical and network data to detect unknown threats. Among the different architectures

provided, we have chosen the following ones that perform the best, as shown in [43]:

1. **Autoencoder:** are unsupervised deep learning models that can learn patterns in data without labeled examples of anomalies [45]. In the context of network traffic analysis, autoencoders effectively capture normal traffic behavior and identify deviations that may indicate attacks or abnormal activities. An AE consists of an encoder and a decoder. The encoder compresses network traffic flows into a lower-dimensional representation, while the decoder reconstructs the original input from this compressed representation. During training, the autoencoder learns to accurately reconstruct normal traffic flows, capturing their inherent patterns. For anomaly detection in network traffic flows, the autoencoder is trained on a dataset containing only normal traffic flows. The model learns to encode and decode these normal patterns, minimizing the reconstruction error. After training, the autoencoder is used to reconstruct new traffic flows, and discrepancies between the reconstructed flows and the original inputs indicate potential anomalies. Higher reconstruction errors suggest deviations from the learned normal behavior.
2. **Autoencoder based on LSTM:** is a variant of the traditional autoencoder is the LSTM-based autoencoder, also known as an autoencoder based on LSTM (Long Short-Term Memory) [46]. The LSTM-based autoencoder leverages the architecture of LSTM networks to specifically address the sequential nature of data and capture temporal dependencies in sequences. While traditional autoencoders process input data as independent and unrelated instances, LSTM-based autoencoders can capture temporal dependencies and patterns in sequential data. This is particularly beneficial for network traffic analysis, where anomalies often exhibit temporal characteristics or evolve over time. By leveraging LSTM cells, the model can effectively capture long-term dependencies and recognize complex temporal behaviors.

3. **Variational Autoencoder:** is a type of autoencoder that is trained with regularization to prevent overfitting [47]. It also ensures that the latent space possesses excellent properties to enable generative processes. The architecture comprises an encoder and a decoder, with the goal of minimizing the reconstruction error. However, in contrast to the standard autoencoder, the variational autoencoder encodes an input as a distribution over the latent space to introduce regularizations in the latent space. During training, the input is first encoded as a distribution over the latent space. Then, a point is randomly sampled from that distribution, the point is decoded, and the reconstruction error is computed and backpropagated throughout the network. In this case, we add the Kullback–Leibler divergence to the reconstruction loss used for the standard autoencoder.
4. **Generative Adversarial Network:** is composed of two adversarial modules: a generator network (also called a generator) and a discriminator network [48,49]. The two modules compete against each other in a way that the discriminator network filters out faux instances of the data, while the generator network is responsible for creating realistic instances and fooling the filtering process. This competitive collaboration teaches the model to generate realistic data. The generator and the discriminator work in tandem, but their roles are adversarial. The generator seeks to improve the quality of its synthetic data to deceive the discriminator, while the discriminator strives to enhance its ability to differentiate between real and synthetic data.

The detailed architectures of the different autoencoders are depicted in Fig. 4, along with the Input and Output shapes ($n \times n$), the number of neurons (n) per layer, and the number of trainable parameters ($\#P$) per layer.

3.3. Threshold selection

Once the model is trained, it is utilized to reconstruct new data samples, and the reconstruction error for each sample is calculated. The reconstruction error can then be thresholded to convert it into a binary classification, determining whether the sample is normal or anomalous.

The threshold can be chosen based on a variety of measures. Three thresholds have been considered in order to convert the anomaly score into a binary value:

T1- P_{99} . The 99° percentile calculated on the reconstruction errors of the validation set.

T2- $\text{median} + 1.5 \times \text{IQR}$. The median is computed on the reconstruction errors of the validation set. The *IQR* (Interquartile Range) is calculated as $(P_{75} - P_{25})$ where P_{75} is the 75° percentile and P_{25} is the 25°.

T3-maximum without outliers. the maximum reconstruction error calculated on the validation set excluding outliers is taken as the threshold. A sample is considered an outlier if its value exceeds the value of $P_{75} + 1.5 \times \text{IQR}$.

The different threshold values are applied to the results in terms of reconstruction error obtained from each different model. The results are evaluated in terms of False Positive Rate (FPR) for the different models and the threshold conducting to the higher value of FPR is selected. Eventually, the best threshold values for each model are used in the detection phase.

4. Testbed and dataset characterization

We carry out an experimental analysis on an existing dataset from a real-world ICS, which represents the intrinsic principles of an actual water treatment plant, as discussed in [4,6]. We describe the testbed used to generate the dataset in Section 4.1, while the dataset itself, as well as the investigated attacks, are described in Section 4.2.

4.1. SWaT testbed

The Secure Water Treatment (SWaT) is a widely adopted testbed for cybersecurity research in the context of CPSs [50]. SWaT is a scaled-down, fully operational raw water purification plant, representing the intrinsic principles of an actual water treatment plant [4]. It employs a modern six-stage architecture for water treatment, including processes like ultrafiltration, de-chlorination, and reverse osmosis. The CPS aspect of SWaT involves PLCs, a layered communications network, HMIs, a SCADA workstation, and a historian. Each stage of SWaT is controlled by a dedicated PLC, which interacts with the physical environment through sensors and actuators connected over a ring network. Sensors measure variables such as water tank levels and flow rates, while actuators operate motorized valves for controlling water inflow. The PLCs execute ladder logic programs, continuously reading sensor data and generating appropriate actuator signals. This data is also accessible to the SCADA system and recorded by the historian for offline analysis. SWaT is an example of the security concerns associated with automating public infrastructure. If the attackers gain access to the network or compromise the PLC programs, they can manipulate the system and make it reach states that cause physical damage, such as tank overflows, pumping empty tanks, or unsafe chemical mixing.

4.2. SWaT dataset and attacks description

In December 2019, a new set of SWaT data was collected, comprising 3 h of normal operation and 1 h during which two attacks were conducted. The dataset is a collection of malware infection attacks against the workstation. The attacks in the dataset include Historian Data Exfiltration and Sensor and Actuator disruption. We utilized this version of the SWaT dataset as it is the only one that includes both network data in the form of raw packet traces and physical data sampled from sensors and actuators during the attacks and normal operation, enabling comprehensive analysis. The first malware attack was triggered by an infected USB stick. It involved 5 min of data exfiltration from the historian server, followed by 10 min of sleep, repeated for 4 cycles. The second malware was downloaded from the Command and Control (C&C) server into the workstation. This attack caused a sensor value to change to an arbitrary value. It included 3 min of disruption and 10 min of sleep, repeated for 5 cycles. As this version of the dataset includes only a few samples of the normal operation of the network which is not enough to capture normal operational patterns, we also use 2015 normal physical data to train models.

Labeling. The dataset was only partially labeled, which means that only the time intervals in which the attacks were conducted are given. A phase of pre-processing and labeling was done for the network and the physical data. For the former, the raw packets were grouped in biflows. Then the flows were labeled as ‘normal’ and ‘malicious’ based on logs and knowledge of the conducted attacks thanks to a network expert. For the latter, the influence of the conducted attacks was found in the sensor data, which were labeled accordingly.

Train–test split. For both network and physical data, the training set consists of data from the normal operation of the infrastructure. For the **network data**, the first hour of capture from the 2019 dataset is used for training. It comprises 442.173 biflows, which allow the system to learn the normal operational patterns. The test set includes data from the different phases of two distinct attacks, consisting of both normal data (flowing during the sleep periods between multiple attack cycles) and anomalous data (flowing during each attack period). Specifically, the test set contains 4,231.290 biflows related to normal operation and 72.127 biflows related to attacks. Of these attack-related biflows, 72.095 are associated with the first attack and 32 with the second attack. No normal biflows occurring right after an attack or in-between different attack phases are included in the training set to avoid contamination from attack behavioral patterns. The remainder of the

dataset is divided into folds for cross-validation. For the **physical data**, the normal operation part of the 2015 dataset, consisting of 396.800 normal samples, is used to train the models, since the 2019 dataset alone was insufficient for learning the normal operational pattern. The test set includes 12.301 normal samples and 901 attack samples from the 2015 dataset. The anomalous samples are all related to the second attack, as the data exfiltration attack does not affect sensor and actuator values.

5. Experimental analysis

The objective of the experimental analysis is to evaluate the various anomaly detection-based models according to the experimental protocol outlined in Section 5.1. Specifically, we examine their ability to detect different cyber-attacks using diverse data sources and compare their performance with various baselines (see Section 5.2). The effectiveness of the models is assessed based on several metrics, defined in Section 3.

5.1. Experimental protocol

The main points of the experimental protocol concern:

1. *Tuning the hyperparameters* for each model and data type (physical and network) to minimize the number of false positives varying Learning Rate, Threshold type, and number of epochs;
2. *Evaluating the effectiveness* of the proposed models when either network data or physical data is provided as input using the best hyperparameters assessed in the previous step, with a focus on the *F1-score* as the evaluation metric;
3. *Comparing* the best approaches emerged in the previous step with respect to the baselines, namely GDN and USAD;
4. *Evaluating the efficiency* of the different models when either network data or physical data is provided as input in terms of *training time*;
5. *Evaluating* the contribution of the different data types (network-physical) in detecting different kinds of cyber-physical attacks.

The experiments have been carried out on Google Colaboratory⁶ using a Virtual Machine with an Intel(R) Xeon CPU @2.20 GHz and 13 GB RAM. The software stack is built on Python 3.9.2. Several libraries have been used for developing DL-based models and the related pipelines (PyTorch⁷ 2.0.1, Keras⁸ 2.8.0, Tensorflow⁹ 2.8.0 and Scikit-learn¹⁰ 1.2.2). To guarantee statistically valid results we performed 5 repetitions of each experiment to derive both the mean and its Confidence Interval (CI) with a significance level $\alpha = 0.05$.

5.2. Baselines

In this section, we describe two different baselines – namely USAD and GDN – that are two of the most frequently used in the context of anomaly detection based on time series [44]. Specifically, we chose these approaches to compare our proposed method to those based on either AE (USAD), or GAN (GDN). We used the hyperparameters recommended by the authors for both models.

- **USAD** [34] is an unsupervised learning approach that works on multivariate time series, where multiple variables are recorded at each time point. The model is trained on a time series dataset consisting only of normal samples and is divided into windows

of size W . The objective is to assign a binary label y to new, unseen time windows. This is done by calculating the anomaly score of a window and comparing it to a predefined threshold. USAD combines two architectures: an autoencoder and a GAN network. It consists of an encoder and two decoders forming two autoencoders. The training process is conducted in two phases. Firstly, both autoencoders are trained to reconstruct normal input data. In the second phase, the autoencoders are trained in an adversarial way using GAN principles. In this phase, AE1 tries to deceive AE2, while the latter tries to distinguish the real data from the data reconstructed by AE1. We trained the model for 100 epochs with a window size $W = 12$ and the output size of the encoder (namely the input size of the decoders) $H = 100$

- **GDN** [36] treats data as multivariate time series and exclusively trains on normal data. The primary aim is to learn the relationships between sensors by employing a graph structure and recognizing deviations from learned patterns. The model comprises the following four main components. (i) Sensor embedding: utilizing embedded vectors, the framework captures the unique characteristics of each sensor, with the objective of creating similar vectors for similar sensors. (ii) Graph structure learning: the model learns a graph whose structure represents the sensor relationships. An oriented graph is created in which an edge from one sensor to another denotes the former's use to model the behavior of the latter. (iii) Graph attention-based forecasting: this component predicts a sensor's future value based on its neighbors' attention function. (iv) Graph deviation scoring: it assigns an anomaly score to each sensor to recognize deviations from learned relationships, hence enabling the identification of the anomaly's source. We trained the model for 30 epochs with a value of $TopK = 2$ which controls the sparsity of the structure of the graph built in the second component.

5.3. Evaluation metrics

In the evaluation of our proposed models, we considered the following metrics instead of accuracy, as the dataset is imbalanced:

- Precision is a metric that measures the proportion of detected anomalies that are truly anomalous (Eq. (1)).

$$Precision = \frac{TP}{(TP + FP)} \quad (1)$$

- Recall measures the proportion of true anomalies that are detected by the model (Eq. (2)). It represents the ability of the model to avoid false negatives or instances in which it fails to detect an anomaly that is present.

$$Recall = \frac{TP}{(TP + FN)} \quad (2)$$

- The F1 score is a metric that combines both precision and recall into a single value. It is calculated as the harmonic mean of precision and recall and provides a way to balance the trade-off between the two metrics (Eq. (3)).

$$F1 - score = \frac{2 * Precision * Recall}{(Precision + Recall)} \quad (3)$$

In anomaly detection, two different F1 scores are used: (i) Total F1 score ($F1_{TOT}$): evaluates overall model performance on the entire dataset, considering both normal and anomalous instances; (ii) F1 for the Anomalous Class ($F1_{AN}$): measures the model's ability to detect anomalies specifically, disregarding its performance on normal instances. The F1 score for the anomalous class is crucial as it focuses solely on the main objective of anomaly detection — accurately identifying anomalies. Both metrics are evaluated for a comprehensive assessment of the model's capabilities.

⁶ <https://colab.research.google.com>

⁷ <https://pytorch.org>

⁸ <https://keras.io>

⁹ <https://www.tensorflow.org/>

¹⁰ <https://scikit-learn.org/>

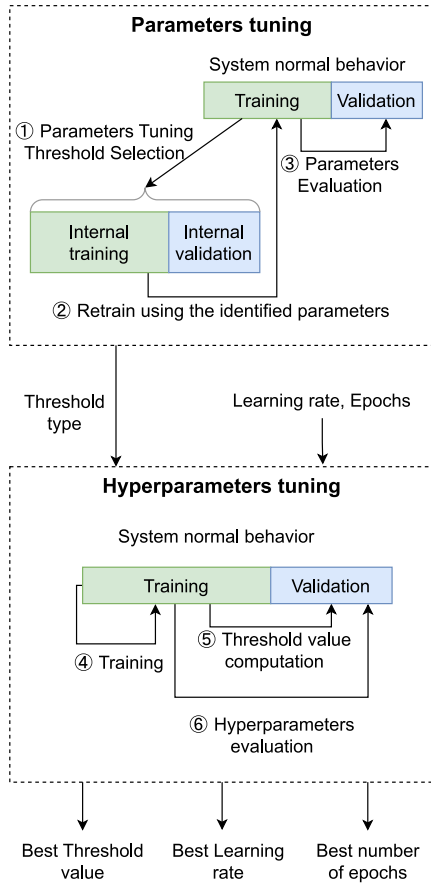


Fig. 5. The training process relies on two main phases. (i) **Parameters Tuning:** dedicated to the selection of the threshold type (parameter) which completes the model to detect anomalies. We use internal training and validation sets obtained by splitting the training set once more to compute the threshold value (①) that is evaluated on the external validation set (③) after training it on the whole training set (②). The output of this phase is the threshold type best suited to each model to produce the least number of FP. (ii) **Hyperparameters Tuning:** dedicated to hyperparameter selection (learning rate, number of epochs, and threshold value). Having fixed the learning rate and the number of epochs, we train the model on the training set (④), then the threshold value is computed on the validation set with the threshold type selected in the previous phase (⑤), and the hyperparameters are evaluated on the same validation set (⑥). The output of this phase is the best combination of hyperparameters for each model.

For all the metrics, mean and Confidence Interval (*CI*) with a significance level $\alpha = 0.05$ are reported. The number of repetitions is 5 for all experiment configurations.

All the metrics mentioned above are computed without using PA, which is commonly employed in anomaly detection models [34]. The PA protocol considers the entire time window (W) as anomalous if at least one instant is detected as an anomaly. However, it has been demonstrated that PA can lead to an overestimation of the detection performance [44]. Therefore, all metrics in our study are computed on a single sample ($W = 1$), which also reduces the detection time.

5.4. Parameter and hyperparameter tuning

We conducted extensive experimentation to enhance the accuracy of the predictions made by the models described in Section 3.2 and to improve their ability to generalize well with unseen data. This process was divided into two phases, as illustrated in Fig. 5. The first phase, the parameter-tuning phase, is detailed in Section 5.4.1, where we explain how the threshold type was selected for each model using an internal training and internal validation dataset. Subsequently, the second

Table 2

Datasets splits where P and N are physical and network data, respectively.

	Internal training	Internal validation	Training	Validation
P	321,408	35,712	357,120	39,680
N	278,568	30,953	309,521	132,652

phase, the hyperparameter-tuning phase, is described in Section 5.4.2, which outputs the best values for the learning rate, number of epochs, and threshold. These hyperparameters are crucial in improving the model's performance and ensuring effective anomaly detection.

To perform the tuning, we divided the dataset describing the system's normal behavior into training and validation sets without shuffling. The split ratio value differed between the network and physical datasets due to the varying number of samples. For the network dataset, a split ratio of 0.3 was chosen to ensure a large training set, while for the physical dataset, a split ratio of 0.1 was selected. In the first phase of tuning, we further split the training set with a value of 0.1 to obtain internal training and validation sets for both the network and physical datasets. The number of samples for each dataset can be seen in Table 2, providing a clear overview of the data distribution.

5.4.1. Parameters tuning

To optimize the parameters of the models, experiments were conducted by following the procedure depicted in the leftmost part of Fig. 5 on the dataset collected under normal conditions. In particular, we split the training set into *internal training* and *internal validation* sets (number ①). The models were trained using the internal training set and the value of the threshold was computed on the internal validation set. Then, we retrained the models over the entire training set (number ②) to evaluate the parameters on the validation set in a final step (number ③). The performance metric used for the parameter tuning phase was the number of False Positives (FP) since the validation set lacked any anomalous instances to avoid the model's specialization to known attack patterns. The outcomes of this process are shown in Table 3. To ensure a reliable and robust assessment, we varied the learning rate, the number of epochs, and the threshold type for each experiment. Specifically three threshold types (i.e., the 99th percentile, the median plus 1.5 times the *IQR*, and the maximum value of anomaly score without outliers defined in Section 3.3) were examined, all of which have been well-documented in prior studies [30,38]. We reported their mean and confidence interval based on five repetitions. This approach guarantees that the final evaluation is based on data that the models have never seen during their training phase, enhancing the reliability of the results. For the physical data part in Table 3, both *AE* and *AE_{LSTM}* present similar results, with the best threshold type being T_1 in all three scenarios. As for the *VAE*, however, the results are extremely unstable and have very wide confidence intervals. Nevertheless, we can identify one scenario that is significantly superior to the others, which is T_3 with the highest learning rate and 20 training epochs. In the case of the *GAN*, the best threshold type is T_1 , which, despite being highly unstable, performs better in at least two out of the three scenarios.

Regarding the network data analysis, the results for *AE* once again indicate that T_1 performs better, consistent with previous findings. However, for *AE_{LSTM}*, the outcomes differ from those observed for the physical data, as T_3 performs better in two out of three scenarios. As for *VAE*, we notice that T_2 and T_3 yield very similar results, but we opted to select T_2 because it has narrower confidence intervals. On the other hand, *GAN* achieves the best results with T_2 in each of the scenarios.

5.4.2. Hyperparameters tuning

Three key hyperparameters were targeted for the optimization of each model: the Learning Rate (*LR*), the Threshold value (T_i), and the number of epochs (*ep.*). *LR* controls the step size during model training, affecting how quickly the model converges. It was systematically varied across a range between $1.00e - 02$ and $1.00e - 04$, commonly reported

Table 3

Parameters tuning. Mean and Confidence Interval of the Number of False Positives (*FP*) on the internal validation set for the different models (*AE*, *AE_{LSTM}*, *VAE*, *GAN*) using different data sources – *Physical Data* and *Network Data* – and varying Learning Rate (*LR*), Threshold type (*T_i*) and epochs (*ep.*). We define *T₁*, *T₂* and *T₃* as *P₉₉*, *P₇₅* + 1.5 × *IQR* and *max* without outliers, respectively.

	Models	AE			AE _{LSTM}			VAE			GAN		
		T _i	T ₁	T ₂	T ₃	T ₁	T ₂	T ₃	T ₁	T ₂	T ₃	T ₁	T ₂
	LR												
Physical Data	1.00E-02	238 ±142.745 (ep. 20)	5,724 ±5,615.582 (ep. 20)	3,632.4 ±3,022.623 (ep. 20)	186.8 ±151.679 (ep. 20)	6,524 ±6,229.302 (ep. 20)	5,718.4 ±5,424.900 (ep. 20)	2,993.8 ±2,950.940 (ep. 50)	16,620 ±12,271.560 (ep. 20)	948.6 ±874.829 (ep. 50)	166 ±89.119 (ep. 50)	172.4 ±104.309 (ep. 50)	39,680 ±0 (20 ep.)
	1.00E-03	238.2 ±185.429 (ep. 20)	4,511.6 ±2,171.208 (ep. 20)	2,741.4 ±1,484.906 (ep. 20)	342.2 ±207.131 (ep. 20)	4,461 ±4,140.242 (ep. 20)	610.8 ±316.215 (ep. 20)	3,134 ±2945.937 (ep. 20)	4,082.8 ±3,635.872 (ep. 20)	3,658.4 ±2,943.048 (ep. 20)	131.6 ±130.021 (ep. 20)	4,719.4 ±3,682.632 (ep. 35)	39,680 ±0 (20 ep.)
	1.00E-04	837 ±811.661 (ep. 35)	5,617.2 ±4,919.945 (ep. 50)	3,382.2 ±2,978.134 (ep. 35)	80.6 ±5.454 (ep. 50)	4,775.4 ±4,722.817 (ep. 35)	727 ±686.060 (ep. 20)	3,191.8 ±3,028.811 (ep. 50)	7,495.2 ±110.051 (ep. 50)	6,190.6 ±365.115 (ep. 50)	5,046 ±4151.357 (ep. 50)	3,772 ±2684.133 (ep. 50)	39,680 ±0 (20 ep.)
	1.00E-02	944.2 ±639.725 (ep. 20)	9,592.4 ±3,260.273 (ep. 35)	6,207 ±1,238.558 (ep. 35)	1,015.6 ±394.795 (ep. 20)	884 ±426.716 (ep. 35)	2,895.6 ±2,026.908 (ep. 20)	943.6 ±784.745 (ep. 20)	482 ±23.697 (ep. 20)	481.3 ±26.562 (ep. 20)	74,569 ±70,964.473 (ep. 50)	431.2 ±39.513 (ep. 50)	132,652 ±0 (20 ep.)
	1.00E-03	1,181.6 ±659.245 (ep. 50)	18,813.8 ±10,050.073 (ep. 50)	7,514.4 ±5,901.635 (ep. 20)	23,165 ±20,191.198 (ep. 20)	16,176.6 ±370.861 (ep. 35)	698.4 ±607.287 (ep. 20)	40,440 ±33,926.614 (ep. 35)	433.2 ±3.213 (ep. 35)	434.6 ±4.695 (ep. 50)	65,253.2 ±56,055.880 (ep. 50)	228.2 ±99.806 (ep. 50)	132,652 ±0 (20 ep.)
	1.00E-04	930.8 ±308.699 (ep. 35)	5,762.4 ±6,819.044 (ep. 35)	932.4 ±453.102 (ep. 35)	3,787.4 ±1,438.585 (ep. 20)	3,248.2 ±123.816 (ep. 20)	457.6 ±20.032 (ep. 20)	822.8 ±632.876 (ep. 35)	492.6 ±7.324 (ep. 35)	475.8 ±18.446 (ep. 50)	17,047.6 ±15,262.519 (ep. 50)	585 ±199.407 (ep. 35)	132,652 ±0 (20 ep.)
Network Data	1.00E-02	944.2 ±639.725 (ep. 20)	9,592.4 ±3,260.273 (ep. 35)	6,207 ±1,238.558 (ep. 35)	1,015.6 ±394.795 (ep. 20)	884 ±426.716 (ep. 35)	2,895.6 ±2,026.908 (ep. 20)	943.6 ±784.745 (ep. 20)	482 ±23.697 (ep. 20)	481.3 ±26.562 (ep. 20)	74,569 ±70,964.473 (ep. 50)	431.2 ±39.513 (ep. 50)	132,652 ±0 (20 ep.)
	1.00E-03	1,181.6 ±659.245 (ep. 50)	18,813.8 ±10,050.073 (ep. 50)	7,514.4 ±5,901.635 (ep. 20)	23,165 ±20,191.198 (ep. 20)	16,176.6 ±370.861 (ep. 35)	698.4 ±607.287 (ep. 20)	40,440 ±33,926.614 (ep. 35)	433.2 ±3.213 (ep. 35)	434.6 ±4.695 (ep. 50)	65,253.2 ±56,055.880 (ep. 50)	228.2 ±99.806 (ep. 50)	132,652 ±0 (20 ep.)
	1.00E-04	930.8 ±308.699 (ep. 35)	5,762.4 ±6,819.044 (ep. 35)	932.4 ±453.102 (ep. 35)	3,787.4 ±1,438.585 (ep. 20)	3,248.2 ±123.816 (ep. 20)	457.6 ±20.032 (ep. 20)	822.8 ±632.876 (ep. 35)	492.6 ±7.324 (ep. 35)	475.8 ±18.446 (ep. 50)	17,047.6 ±15,262.519 (ep. 50)	585 ±199.407 (ep. 35)	132,652 ±0 (20 ep.)
	1.00E-02	944.2 ±639.725 (ep. 20)	9,592.4 ±3,260.273 (ep. 35)	6,207 ±1,238.558 (ep. 35)	1,015.6 ±394.795 (ep. 20)	884 ±426.716 (ep. 35)	2,895.6 ±2,026.908 (ep. 20)	943.6 ±784.745 (ep. 20)	482 ±23.697 (ep. 20)	481.3 ±26.562 (ep. 20)	74,569 ±70,964.473 (ep. 50)	431.2 ±39.513 (ep. 50)	132,652 ±0 (20 ep.)
	1.00E-03	1,181.6 ±659.245 (ep. 50)	18,813.8 ±10,050.073 (ep. 50)	7,514.4 ±5,901.635 (ep. 20)	23,165 ±20,191.198 (ep. 20)	16,176.6 ±370.861 (ep. 35)	698.4 ±607.287 (ep. 20)	40,440 ±33,926.614 (ep. 35)	433.2 ±3.213 (ep. 35)	434.6 ±4.695 (ep. 50)	65,253.2 ±56,055.880 (ep. 50)	228.2 ±99.806 (ep. 50)	132,652 ±0 (20 ep.)
	1.00E-04	930.8 ±308.699 (ep. 35)	5,762.4 ±6,819.044 (ep. 35)	932.4 ±453.102 (ep. 35)	3,787.4 ±1,438.585 (ep. 20)	3,248.2 ±123.816 (ep. 20)	457.6 ±20.032 (ep. 20)	822.8 ±632.876 (ep. 35)	492.6 ±7.324 (ep. 35)	475.8 ±18.446 (ep. 50)	17,047.6 ±15,262.519 (ep. 50)	585 ±199.407 (ep. 35)	132,652 ±0 (20 ep.)

Table 4

Hyperparameters tuning. Best hyperparameters for each model.

	Models	Epochs	Learning rate	Threshold value
Physical data	AE	20	1e-02	0.3249 (<i>T₁</i>)
	<i>AE_{LSTM}</i>	50	1e-04	0.4168 (<i>T₁</i>)
	VAE	20	1e-02	0.3519 (<i>T₃</i>)
	GAN	20	1e-03	0.5144 (<i>T₁</i>)
Network data	AE	50	1e-03	1.0880 (<i>T₁</i>)
	<i>AE_{LSTM}</i>	20	1e-04	0.5399 (<i>T₃</i>)
	VAE	20	1e-03	0.5980 (<i>T₂</i>)
	GAN	50	1e-03	0.5098 (<i>T₂</i>)

in the existing literature. The threshold value is the decision boundary used to classify data as normal or anomalous. Tuning the threshold is essential to balance *FPs* and *FNs*. The optimal number of epochs for each model was also taken into account during the hyperparameter tuning process. To determine this, a range of epochs (20, 35, 50) was explored. The hyperparameter tuning process is depicted in the rightmost part of Fig. 5. After selecting the threshold type in the previous phase, we explore every combination of hyperparameter values (i.e., *LR* and *ep.*) we have fixed for network and sensor data. Given a set of hyperparameters, we train each model on the training set (number ①). The threshold value is then computed on the validation set (number ②), and the set of hyperparameters is evaluated on the same validation set (number ③). The optimal hyperparameters, including the Learning Rate (*LR*), Threshold value (*T_i*), and number of epochs (*ep.*) for each model, pertaining to both network and sensor data, are reported in Table 4. Regarding the physical data, *AE* and *VAE* exhibit the same optimal learning rates and number of epochs, while their threshold values are very close. On the other hand, *AE_{LSTM}* achieves superior results with the slowest learning rate along with the highest number of epochs. As for *GAN*, it yields the highest threshold values, almost reaching the critical threshold of 0.5, which would result in a balanced probability of recognizing anomalies. Regarding the network data, the best hyperparameters for *AE* and *VAE* diverge. The former performs better with 50 epochs, a learning rate of $1e-03$, and a very high threshold value. On the other hand, the latter achieves optimal results with 20 epochs and, although using the same learning rate, requires a much lower threshold. As for *AE_{LSTM}*, it converges to the best results with the lowest number of epochs and the slowest learning rate value, and its threshold value is near 0.5, similar to *GAN*, which achieves its best results with 50 epochs and a learning rate of $1e-03$.

Table 5

Efficiency evaluation. Training time – mean and confidence interval – in seconds for all models – with the best hyperparameters – using Physical and Network data sources.

	Physical	Network
USAD	323.841 ± 12.592	281.124 ± 6.683
GDN	2,072.345 ± 1,466.391	3,203.766 ± 1.344
AE	28.293 ± 10.921	27.387 ± 2.438
<i>AE_{LSTM}</i>	27.124 ± 2.795	15.258 ± 0.394
VAE	23.481 ± 1.807	27.569 ± 3.135
GAN	620.472 ± 42.458	1,125.456 ± 130.672

6. Results

In this section, we discuss the efficiency and effectiveness of the proposed framework, which are detailed in Sections 6.1 and 6.2, respectively. We further compare the proposal w.r.t. several baselines, also comparing the computed threshold with the optimal one (see Section 6.3).

6.1. Efficiency evaluation

Table 5 presents the training times in seconds for each model and data type. We show the results for the models with the best hyperparameters chosen in the previous section. The reported values represent the mean and 95% confidence interval based on 5 experiments.

Notably, *GDN* exhibits a significantly greater training time for both network and sensor data compared to all the other models. On the other hand, *AE*, *AE_{LSTM}*, and *VAE* demonstrate comparable results in terms of training time. Moreover, they stand out as the least time-consuming models for both physical and sensor data while exhibiting similar performance.

While the training times of *USAD* and *GAN* are higher compared to the Autoencoder models, their results are significantly better than those of *GDN*.

Eventually, we highlight that the most efficient models for both physical and network data are respectively *VAE* and *AE_{LSTM}*, as emphasized in bold.

6.2. Performance evaluation

The different models with the best hyperparameters selected in Section 5.4 are evaluated on network and physical data using the

Table 6

Performance Evaluation I (Results in terms of mean and confidence interval for $F1_{TOT}$, $F1_{AN}$, Precision, Recall) - Results on the test set for the *first attack*, considering *Network Data* with the best hyperparameters for each model.

Method	F1-total	F1-anomaly	Precision	Recall
USAD	0.555 $\pm$ 0.007	0.128 $\pm$ 0.001	0.534 $\pm$ 0.000	0.578 $\pm$ 0.004
GDN	0.755 $\pm$ 0.001	–	0.682 $\pm$ 0.020	0.835 $\pm$ 0.003
AE	0.939 $\pm$ 0.018	0.886 $\pm$ 0.035	0.898 $\pm$ 0.028	0.992 $\pm$ 0.002
AE _{LSTM}	0.485 $\pm$ 0.002	0.0006 $\pm$ 0.0004	0.474 $\pm$ 0.004	0.498 $\pm$ 0.001
VAE	0.484 $\pm$ 0.002	0.0007 $\pm$ 0.0004	0.474 $\pm$ 0.004	0.498 $\pm$ 0.001
GAN	0.485 $\pm$ 0.001	0	0.471 $\pm$ 0.003	0.5

Table 7

Performance Evaluation II (Results in terms of mean and confidence interval for $F1_{TOT}$, $F1_{AN}$, Precision, Recall) on the test set for the *second attack*, considering *Network Data* with the best hyperparameters for each model.

Model	F1-total	F1-anomaly	Precision	Recall
USAD	0.488 $\pm$ 0.007	0.0005 $\pm$ 0.000	0.500 $\pm$ 0.000	0.476 $\pm$ 0.005
GDN	0.001 $\pm$ 0.0005	–	0.001 $\pm$ 0.000	0.517 $\pm$ 0.177
AE	0.497 $\pm$ 0.003	0.003 $\pm$ 0.002	0.501 $\pm$ 0.0005	0.891 $\pm$ 0.280
AE _{LSTM}	0.498 $\pm$ 0.002	0.0005 $\pm$ 0.001	0.500 $\pm$ 0.0003	0.513 $\pm$ 0.047
VAE	0.498 $\pm$ 0.002	0	0.500 $\pm$ 0.001	0.513 $\pm$ 0.047
GAN	0.500 $\pm$ 0.001	0	0.500 $\pm$ 0.001	0.500 $\pm$ 0.001

test set. The evaluation metrics used are $F1_{TOT}$, $F1_{AN}$, Precision, and Recall, and the results are reported as mean and 95% confidence interval based on 5 experiments.

For the evaluation with network data, the results are presented in [Table 6](#) for the first attack and [Table 7](#) for the second attack. As for the evaluation with sensor data as input, the results are entirely shown in [Table 8](#). The entire test set is considered without distinguishing between the first and second attacks since only the first attack impacts sensor values.

In [Table 6](#), it is shown that the best-performing model in detecting the first attack for all considered metrics is *AE*, outperforming both the baselines and the other models. However, in [Table 7](#), all models using network data perform poorly, failing to recognize the second attack. This is likely due to the limited impact of the second attack on the network data. Nevertheless, *AE* remains the best model for network data, exhibiting comparable values of *F1* with *AE_{LSTM}* and *VAE*, but significantly higher Recall.

In [Table 8](#), the best-performing model with Physical Data is *GAN*, achieving superior performance across all the considered metrics. Additionally, *AE*, *AE_{LSTM}*, and *VAE* also obtain good results, performing better than the baselines in terms of *F1*-total.

The evaluation results indicate that no single model performs well across all data types. Specifically, the *AE* model obtains the best performance for Network Data. On the other hand, the *GAN* model emerges as the top performer for Physical Data, showcasing superior performance across all evaluation metrics. However, it is worth noting that the *GAN* model's strengths in handling Physical Data are not mirrored in Network Data evaluation. For Network Data, the *GAN* model performs poorly, failing to correctly classify any instances of anomalies for both attack scenarios ($F1 - anomaly = 0$).

6.3. Benchmarking against the optimal solution

The best threshold is computed using the ROC curve on the test set. The ROC curve method takes the reconstructed errors as input and computes – for each possible threshold value – the TPR (True Positive Rate) and TNR (True Negative Rate). The optimal threshold is chosen by selecting the one that leads to the point on the curve that maximizes Youden's J statistic (Youden's Index), which is given by:

$$J = TPR + TNR - 1 \quad (4)$$

Table 8

Performance Evaluation III (Results in terms of mean for $F1_{TOT}$, $F1_{AN}$, Precision, Recall) on the entire test set considering *Physical Data* with the best hyperparameters for each model. The confidence interval is also specified with a significance level of α (0.95).

Model	F1-total	F1-anomaly	Precision	Recall
USAD	0.562 $\pm$ 0.160	0.31 $\pm$ 0.186	0.586 $\pm$ 0.089	0.768 $\pm$ 0.291
GDN	0.492 $\pm$ 0.263	–	0.349 $\pm$ 0.206	0.917 $\pm$ 0.144
AE	0.658 $\pm$ 0.232	0.397 $\pm$ 0.303	0.643 $\pm$ 0.130	0.734 $\pm$ 0.224
AE _{LSTM}	0.563 $\pm$ 0.132	0.240 $\pm$ 0.200	0.589 $\pm$ 0.155	0.600 $\pm$ 0.081
VAE	0.645 $\pm$ 0.177	0.428 $\pm$ 0.380	0.612 $\pm$ 0.147	0.718 $\pm$ 0.262
GAN	0.797 $\pm$ 0.001	0.638 $\pm$ 0.002	0.734 $\pm$ 0.001	0.958 $\pm$ 0.0003

Table 9

Results of the best models over a single run with our threshold and the optimal threshold computed with the ROC curve method.

Model	T	$F1_{tot}$	$F1_{An}$	Precision	Recall
GAN	non optimal	0.80	0.64	0.74	0.96
GAN	optimal	0.80	0.64	0.74	0.96
AE	non optimal	0.93	0.87	0.89	0.99
AE	optimal	0.93	0.87	0.88	1.00

This threshold balances the trade-off between recall and specificity and provides the best overall performance for the model.

We compare the results obtained by our model over a single experiment with the threshold computed by our hyperparameter tuning framework on the validation set for the best-performing models for each data source, as well as with the results obtained with the same models but selecting the optimal threshold on the test set. The results in [Table 9](#) demonstrate that the performance in terms of all the considered metrics is never worse compared to the ones obtained by the models using the optimal threshold.

6.4. Discussion

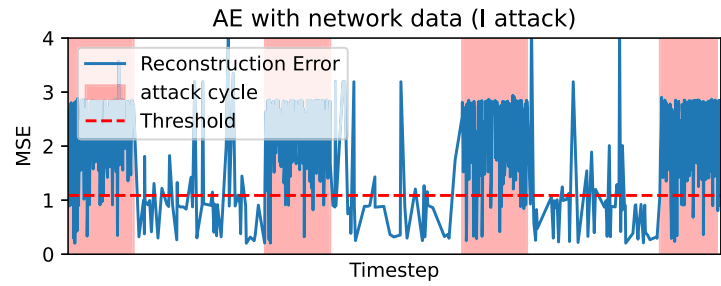
The obtained results demonstrate a significant improvement compared to the baselines, both in terms of efficiency (training time) and effectiveness (evaluation metrics). However, it is evident that there is no single model that performs well for both data types. Specifically, *GAN* excels only with physical data, while *AE* performs well exclusively with network data.

AE also achieves favorable results in efficiency analysis, with low training times, which can be advantageous when retraining the model becomes necessary. This outcome is also confirmed in other application domains such as the Internet of Things (IoT), where *AE*-based approaches (see [39] for instance) have yielded promising results. However, although [39] designs an *AE* model using data from the application layer to identify anomalies in IoT networks, our analysis is based on investigating deep learning models to harvest sensor/actuator data modeled as multivariate time series and biflows inferred from network traffic, that are the main relevant sources in CPSs, as underlined in [4,6]

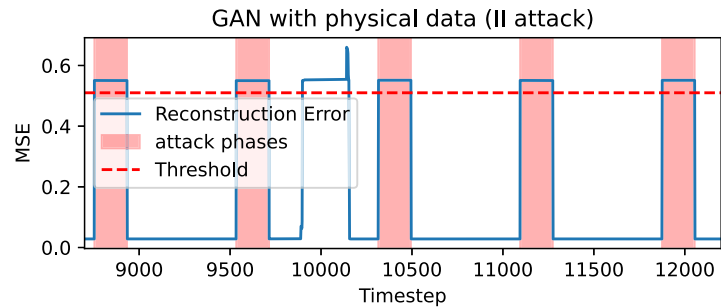
The results obtained using the best suited models for network and physical data, with the hyperparameters selected during the tuning phase, reveal that no single model is capable of detecting all of the attacks.

Although *GAN* exhibits excellent performance (as shown in 6(b)), it can only detect the second attack since it operates on physical data that remains unaffected by the first attack. Similarly, *AE* effectively identifies the first attack using network data (as shown in 6(b)), but its capability to detect the second attack is limited to just 2 biflows, given its scarce impact on network data.

Furthermore, there is no superior data type for attack detection, and it is beneficial to consider both types for comprehensive analysis. The use of both network and physical data can provide a more robust approach to detect various types of attacks effectively.



(a) Reconstruction Error obtained with **AE** on *network data* over the **first** Attack cycles.



(b) Reconstruction Error obtained with **GAN** on *physical data* over the **second** Attack cycles.

Fig. 6. Performance of the different models over the different attack and data types.

7. Conclusion

In conclusion, this paper is focused on the analysis of different anomaly detection models, including *AE*, *AE_{LSTM}*, *VAE*, and *GAN* evaluated on both network data (in the form of bidirectional flows) and physical (both sensor and actuator) data. To this end, a novel dataset that presents a simultaneous collection of network traffic and physical data has been used, and the models have been fine-tuned and evaluated on various thresholds and input data. The results of our evaluation indicated that different models perform better on different data types. After selecting the best model for the network and physical data, we compared their performance in detecting two types of cyber-physical attacks. It was observed that the network model could successfully detect the first attack entirely and identify a few biflows of the second attack. However, the physical data model could only detect the second attack. These findings highlight the importance of using a comprehensive approach that combines both network and physical data to enhance the detection capabilities and overall resilience of the system against various cyber-physical attacks.

Since anomaly detection performed on certain attacks does not guarantee good results in a real-world scenario, as highlighted in [51], in future research we plan to develop a unified anomaly detection framework that takes both network and physical data as input, providing a single anomaly result. The goal is to improve the system's ability to detect different types of complex attacks and provide timely responses. This will involve studying how to combine different anomaly detection techniques and process network and physical datasets together to create a model that considers all possible attack scenarios.

By addressing these challenges and further developing the proposed unified anomaly detection framework, we aim to make significant strides in enhancing the security and resilience of CPSs and critical infrastructures against cyber-physical attacks.

CRediT authorship contribution statement

Roberto Canonico: Writing – original draft, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Giovanni Esposito:** Writing – original draft, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Annalisa Navarro:** Writing – original draft, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Simon Pietro Romano:** Writing – original draft, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Giancarlo Sperli:** Writing – original draft, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Andrea Vignali:** Writing – original draft, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work has been funded by the project NextGenerationEU via PNRR - DM 352 (CUP: E66G22000400009) and it was partially supported by project cyberHuman, part of the SERICS program (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU.

Data availability

The authors do not have permission to share data.

References

- [1] C. Konstantinou, O.M. Anubi, Resilient cyber-physical energy systems using prior information based on Gaussian process, *IEEE Trans. Ind. Inform.* 18 (3) (2022) 2160–2168, <http://dx.doi.org/10.1109/TII.2021.3099974>.
- [2] A. Goknil, P. Nguyen, S. Sen, D. Politaki, H. Nivais, K.J. Pedersen, A. Suyuthi, A. Anand, A. Ziegenbein, A systematic review of data quality in CPS and IoT for industry 4.0, *ACM Comput. Surv.* 55 (14s) (2023) <http://dx.doi.org/10.1145/3593043>.
- [3] J. Giraldo, D. Urbina, A. Cardenas, J. Valente, M. Faisal, J. Ruths, N.O. Tippenhauer, H. Sandberg, R. Candell, A survey of physics-based attack detection in cyber-physical systems, *ACM Comput. Surv.* 51 (4) (2018) <http://dx.doi.org/10.1145/3203245>.
- [4] Y. Luo, Y. Xiao, L. Cheng, G. Peng, D.D. Yao, Deep learning-based anomaly detection in cyber-physical systems: Progress and opportunities, *ACM Comput. Surv.* 54 (5) (2021) <http://dx.doi.org/10.1145/3453155>.
- [5] D.C. Le, N. Zincir-Heywood, Anomaly detection for insider threats using unsupervised ensembles, *IEEE Trans. Netw. Serv. Manag.* 18 (2) (2021) 1152–1164, <http://dx.doi.org/10.1109/TNSM.2021.3071928>.
- [6] R. Canonico, G. Sperli, Industrial cyber-physical systems protection: A methodological review, *Comput. Secur.* 135 (2023) 103531, <http://dx.doi.org/10.1016/j.cose.2023.103531>.
- [7] H. Kivrak, M.Z. Karakusak, S. Watson, B. Lennox, Cyber-physical system architecture of autonomous robot ecosystem for industrial asset monitoring, *Comput. Commun.* 218 (2024) 72–84, <http://dx.doi.org/10.1016/j.comcom.2024.02.013>.
- [8] I. Zografopoulos, J. Ospina, X. Liu, C. Konstantinou, Cyber-physical energy systems security: Threat modeling, risk assessment, resources, metrics, and case studies, *IEEE Access* 9 (2021) 29775–29818, <http://dx.doi.org/10.1109/ACCESS.2021.3058403>.
- [9] B. Hammi, S. Zeadally, J. Nebhen, Security threats, countermeasures, and challenges of digital supply chains, *ACM Comput. Surv.* (2023) <http://dx.doi.org/10.1145/3588999>.
- [10] M.M. Cook, A.K. Marnerides, D. Pezaros, PLCPrint: Fingerprinting memory attacks in programmable logic controllers, *IEEE Trans. Inf. Forensics Secur.* 18 (2023) 3376–3387, <http://dx.doi.org/10.1109/TIFS.2023.3277688>.
- [11] A. Al-Abassi, H. Karimipour, A. Dehghantanha, R.M. Parizi, An ensemble deep learning-based cyber-attack detection in industrial control system, *IEEE Access* 8 (2020) 83965–83973, <http://dx.doi.org/10.1109/ACCESS.2020.2992249>.
- [12] A.T.A. Ghazo, R. Kumar, Critical attacks set identification in attack graphs for computer and SCADA/ICS networks, *IEEE Trans. Syst. Man, Cybern. Syst.* (2023) 1, <http://dx.doi.org/10.1109/TSMC.2023.3274613>.
- [13] M. Barrère, C. Hankin, D. O'Reilly, Cyber-physical attack graphs (CPAGs): Composable and scalable attack graphs for cyber-physical systems, *Comput. Secur.* 132 (2023) 103348, <http://dx.doi.org/10.1016/j.cose.2023.103348>.
- [14] Q. Gu, D. Formby, S. Ji, B. Saltaformaggio, A. Bourgeois, R. Beyah, This hacker knows physics: Device physics aware mimicry attacks in cyber-physical systems, *IEEE Trans. Dependable Secur. Comput.* 19 (5) (2022) 3218–3230, <http://dx.doi.org/10.1109/TDSC.2021.3089163>.
- [15] J. Tian, B. Wang, J. Li, C. Konstantinou, Datadriven false data injection attacks against cyber-physical power systems, *Comput. Secur.* 121 (2022) 102836, <http://dx.doi.org/10.1016/j.cose.2022.102836>.
- [16] S. Kim, K.-J. Park, C. Lu, A survey on network security for cyber-physical systems: From threats to resilient design, *IEEE Commun. Surv. Tutor.* 24 (3) (2022) 1534–1573, <http://dx.doi.org/10.1109/COMST.2022.3187531>.
- [17] R. Soni, B. Mehta, Diagnosis and prognosis of incipient faults and insulation status for asset management of power transformer using fuzzy logic controller & fuzzy clustering means, *Electr. Power Syst. Res.* 220 (2023) 109256, <http://dx.doi.org/10.1016/j.epsr.2023.109256>.
- [18] R. Dagnas, M. Barbeau, M. Boutin, J. Garcia-Alfaro, R. Yaich, Exploring the quantitative resilience analysis of cyber-physical systems, in: 2023 IFIP Networking Conference, IFIP Networking, IEEE, 2023, pp. 1–6, <http://dx.doi.org/10.23919/IFIPNetworking57963.2023.10186355>.
- [19] G. Pang, C. Shen, L. Cao, A.V.D. Hengel, Deep learning for anomaly detection: A review, *ACM Comput. Surv.* 54 (2) (2021) <http://dx.doi.org/10.1145/3439950>.
- [20] M. Cook, A. Marnerides, C. Johnson, D. Pezaros, A survey on industrial control system digital forensics: Challenges, advances and future directions, *IEEE Commun. Surv. Tutor.* (2023) 1, <http://dx.doi.org/10.1109/COMST.2023.3264680>.
- [21] S. Lyu, K. Wang, L. Zhang, B. Wang, Process-oriented heterogeneous graph learning in GNN-based ICS anomalous pattern recognition, *Pattern Recognit.* 141 (2023) 109661, <http://dx.doi.org/10.1016/j.patcog.2023.109661>.
- [22] D. Bhamare, M. Zolanvari, A. Erbad, R. Jain, K. Khan, N. Meskin, Cybersecurity for industrial control systems: A survey, *Comput. Secur.* 89 (2020) 101677, <http://dx.doi.org/10.1016/j.cose.2019.101677>.
- [23] M. Kravchik, A. Shabtai, Efficient cyber attack detection in industrial control systems using lightweight neural networks and pca, *IEEE Trans. Dependable Secur. Comput.* 19 (4) (2021) 2179–2197, <http://dx.doi.org/10.1109/TDSC.2021.3050101>.
- [24] M. Azzam, L. Pasquale, G. Provan, B. Nuseibeh, Forensic readiness of industrial control systems under stealthy attacks, *Comput. Secur.* 125 (2023) 103010, <http://dx.doi.org/10.1016/j.cose.2022.103010>.
- [25] Q. Zhang, A.Z. Mohammed, Z. Wan, J.-H. Cho, T.J. Moore, Diversity-by-design for dependable and secure cyber-physical systems: A survey, *IEEE Trans. Netw. Serv. Manag.* 19 (1) (2022) 706–728, <http://dx.doi.org/10.1109/TNSM.2021.3091391>.
- [26] L. Cheng, K. Tian, D.D. Yao, L. Sha, R.A. Beyah, Checking is believing: Event-aware program anomaly detection in cyber-physical systems, *IEEE Trans. Dependable Secur. Comput.* 18 (2) (2019) 825–842, <http://dx.doi.org/10.1109/TDSC.2019.2906161>.
- [27] W.T. Lunardi, M.A. Lopez, J.-P. Giacalone, ARCADE: Adversarially regularized convolutional autoencoder for network anomaly detection, *IEEE Trans. Netw. Serv. Manag.* 20 (2) (2023) 1305–1318, <http://dx.doi.org/10.1109/TNSM.2022.3229706>.
- [28] Y. Yang, Y. Yan, Z. Gao, L. Rui, R. Lyu, B. Gao, P. Yu, A network traffic classification method based on dual-mode feature extraction and hybrid neural networks, *IEEE Trans. Netw. Serv. Manag.* (2023) 1, <http://dx.doi.org/10.1109/TNSM.2023.3262246>.
- [29] F. Erlacher, F. Dressler, On high-speed flow-based intrusion detection using snort-compatible signatures, *IEEE Trans. Dependable Secur. Comput.* 19 (1) (2022) 495–506, <http://dx.doi.org/10.1109/TDSC.2020.2973992>.
- [30] B. Lindemann, B. Maschler, N. Sahlab, M. Weyrich, A survey on anomaly detection for technical systems using LSTM networks, *Comput. Ind.* 131 (2021) 103498, <http://dx.doi.org/10.1016/j.compind.2021.103498>.
- [31] J. Singh, M. Wazid, A.K. Das, V. Chamola, M. Guizani, Machine learning security attacks and defense approaches for emerging cyber physical applications: A comprehensive survey, *Comput. Commun.* 192 (2022) 316–331, <http://dx.doi.org/10.1016/j.comcom.2022.06.012>.
- [32] A.R. Bahlali, A. Bachir, Machine learning anomaly-based network intrusion detection: Experimental evaluation, in: International Conference on Advanced Information Networking and Applications, Springer, 2023, pp. 392–403, http://dx.doi.org/10.1007/978-3-031-28451-9_34.
- [33] M. Conti, D. Donadel, F. Turrin, A survey on industrial control system testbeds and datasets for security research, *IEEE Commun. Surv. Tutor.* 23 (4) (2021) 2248–2294, <http://dx.doi.org/10.1109/COMST.2021.3094360>.
- [34] J. Audibert, P. Michiardi, F. Guyard, S. Marti, M.A. Zuluaga, Usad: Unsupervised anomaly detection on multivariate time series, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020, pp. 3395–3404, <http://dx.doi.org/10.1145/3394486.3403392>.
- [35] K.O. Akpinar, I. Ozelik, Methodology to determine the device-level periodicity for anomaly detection in ethernet-based industrial control network, *IEEE Trans. Netw. Serv. Manag.* 18 (2) (2021) 2308–2319, <http://dx.doi.org/10.1109/TNSM.2020.3037050>.
- [36] A. Deng, B. Hooi, Graph neural network-based anomaly detection in multivariate time series, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, No. 5, 2021, pp. 4027–4035, <http://dx.doi.org/10.1609/aaai.v35i5.16523>.
- [37] C. Sheng, Y. Yao, W. Li, W. Yang, Y. Liu, Unknown attack traffic classification in SCADA network using heuristic clustering technique, *IEEE Trans. Netw. Serv. Manag.* (2023) 1, <http://dx.doi.org/10.1109/TNSM.2023.3238402>.
- [38] C. Tang, L. Xu, B. Yang, Y. Tang, D. Zhao, GRU-based interpretable multivariate time series anomaly detection in industrial control system, *Comput. Secur.* 127 (2023) 103094, <http://dx.doi.org/10.1016/j.cose.2023.103094>.
- [39] A. Abusitta, G.H. de Carvalho, O.A. Wahab, T. Halabi, B.C. Fung, S.A. Mamoori, Deep learning-enabled anomaly detection for IoT systems, *Internet Things* 21 (2023) 100656, <http://dx.doi.org/10.1016/j.iot.2022.100656>.
- [40] G. Bovenzi, G. Aceto, D. Ciunzio, A. Montieri, V. Persico, A. Pescapé, Network anomaly detection methods in IoT environments via deep learning: A fair comparison of performance and robustness, *Comput. Secur.* 128 (2023) 103167, <http://dx.doi.org/10.1016/j.cose.2023.103167>.
- [41] P. Bellavista, N. Biccocchi, M. Fogli, C. Giannelli, M. Mamei, M. Picone, Requirements and design patterns for adaptive, autonomous, and context-aware digital twins in industry 4.0 digital factories, *Comput. Ind.* 149 (2023) 103918, <http://dx.doi.org/10.1016/j.compind.2023.103918>.
- [42] M. Alanazi, A. Mahmood, M.J.M. Chowdhury, SCADA vulnerabilities and attacks: A review of the state-of-the-art and open issues, *Comput. Secur.* 125 (2023) 103028, <http://dx.doi.org/10.1016/j.cose.2022.103028>.
- [43] G. Li, J.J. Jung, Deep learning for anomaly detection in multivariate time series: Approaches, applications, and challenges, *Inf. Fusion* 91 (2023) 93–102, <http://dx.doi.org/10.1016/j.inffus.2022.10.008>.
- [44] S. Kim, K. Choi, H.-S. Choi, B. Lee, S. Yoon, Towards a rigorous evaluation of time-series anomaly detection, *Proc. AAAI Conf. Artif. Intell.* 36 (7) (2022) 7194–7201, <http://dx.doi.org/10.1609/aaai.v36i7.20680>.
- [45] A. Chevrot, A. Vernotte, B. Legeard, CAE: Contextual auto-encoder for multivariate time-series anomaly detection in air transportation, *Comput. Secur.* 116 (2022) 102652, <http://dx.doi.org/10.1016/j.cose.2022.102652>.
- [46] M. Said Elsayed, N.-A. Le-Khac, S. Dev, A.D. Jurcut, Network anomaly detection using LSTM based autoencoder, in: Proceedings of the 16th ACM Symposium on QoS and Security for Wireless and Mobile Networks, in: Q2SWinet '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 37–45, <http://dx.doi.org/10.1145/3416013.3426457>.

- [47] R. Alguliyev, L. Sukhostat, A. Mammadov, Anomaly detection in cyber-physical systems based on BiGRU-VAE, in: 2022 IEEE 16th International Conference on Application of Information and Communication Technologies, AICT, IEEE, 2022, pp. 1–5, <http://dx.doi.org/10.1109/AICT55583.2022.10013581>.
- [48] D. Li, D. Chen, J. Goh, S.-k. Ng, Anomaly detection with generative adversarial networks for multivariate time series, 2018, arXiv preprint [arXiv:1809.04758](https://arxiv.org/abs/1809.04758).
- [49] E. Brophy, Z. Wang, Q. She, T. Ward, Generative adversarial networks in time series: A systematic literature review, *ACM Comput. Surv.* 55 (10) (2023) <http://dx.doi.org/10.1145/3559540>.
- [50] J. Goh, S. Adepu, K.N. Junejo, A. Mathur, A dataset to support research in the design of secure water treatment systems, in: *Critical Information Infrastructures Security: 11th International Conference, CRITIS 2016, Paris, France, October 10–12, 2016, Revised Selected Papers 11*, Springer, 2017, pp. 88–99.
- [51] C. Wang, K. Wu, T. Zhou, G. Yu, Z. Cai, TSAGen: Synthetic time series generation for KPI anomaly detection, *IEEE Trans. Netw. Serv. Manag.* 19 (1) (2022) 130–145, <http://dx.doi.org/10.1109/TNSM.2021.3098784>.